Part 1: Randomized Quicksort Analysis

Stephanie Peguero

1. Implementation

To evaluate the efficiency of Quicksort, two variations of the algorithm were implemented: Randomized Quicksort and Deterministic Quicksort. Both implementations utilize list comprehensions to cleanly partition the array into three sub-arrays: left (elements less than the pivot), middle (elements equal to the pivot), and right (elements greater than the pivot).

The primary difference between the two lies in the pivot selection. The deterministic version strictly selects the first element (arr[0]), while the randomized version uses Python's built-in random.choice(arr) to select a pivot uniformly at random from the current subarray.

This implementation efficiently handles edge cases. Creating a separate middle list for elements equal to the pivot prevents unnecessary recursive calls on duplicate values. Empty arrays or arrays with a single element are handled via a base case condition (if len(arr) <= 1: return arr).

2. Analysis

The average-case time complexity of Randomized Quicksort is $O(n \log n)$. This efficiency is achieved because selecting a pivot uniformly at random makes it highly probable that the pivot divides the array into relatively balanced partitions. Even an unbalanced split (such as 25% to the left and 75% to the right) reduces the problem size by a constant fraction, which results in a recursion tree of depth $O(\log n)$. At each level of the tree, the partitioning process takes $O(n)$ time, yielding an expected total time of $O(n \log n)$.

This average-case complexity can be proven using indicator random variables. Let X_{ij} be an indicator random variable that equals 1 if the i-th smallest element and the j-th smallest element are compared during the execution of the algorithm, and 0 otherwise. The probability of comparing these two elements is $\frac{2}{j-i+1}$, because a comparison only occurs if either the i-th or j-th element is chosen as the pivot before any element between them. Summing these expected values over all possible pairs of elements yields an expected total number of comparisons bounded by $O(n \log n)$.

Conversely, the deterministic version is vulnerable to a worst-case scenario $O(n^2)$ when given an already sorted or reverse-sorted array. Because it consistently picks the first element, an

already sorted array causes 0 elements to go to the left partition and $n - 1$ elements to go to the right. This extreme imbalance can be expressed through the following recurrence relation:

$$T(n) = T(n - 1) + O(n)$$

When expanded layer by layer, the total work required is the sum of the first n integers:

$$n + (n - 1) + (n - 2) + \ldots + 1 = \frac{n(n+1)}{2} = \frac{1}{2}n^2 + \frac{1}{2}n$$

Dropping the constants and lower-order terms yields a worst-case time complexity of $O(n^2)$.

3. Comparison

To empirically test these algorithms, I developed a script to measure their execution time across four distinct array distributions (size $n = 1000$). To accurately capture the worst-case behavior of the deterministic version, it was necessary to adjust the environment's recursion limit using sys.setrecursionlimit(2000). According to the Python Software Foundation (n.d.), the default recursion limit is intentionally restricted to prevent infinite recursion from crashing the C stack. Without this adjustment, the deterministic algorithm triggered a RecursionError because peeling off exactly one element per recursive call forces the algorithm to exceed the default stack limit of 1,000 frames.

Empirical Results (Size $n = 1000$**):**

Array Distribution	**Randomized Quicksort**	**Deterministic Quicksort**
Random Array	0.00165 seconds	0.00440 seconds
Sorted Array	0.00164 seconds	0.05862 seconds
Reverse Sorted Array	0.00159 seconds	0.10894 seconds
Repeated Array	0.00038 seconds	0.00024 seconds

Discussion of Results: The empirical data aligns perfectly with the theoretical analysis. The Deterministic Quicksort slowed down significantly on the Sorted and Reverse Sorted arrays (0.058s and 0.108s, respectively), proving the mathematical reality of the $O(n^2)$ worst-case recurrence relation. The Randomized Quicksort maintained its $O(n \log n)$ speed across these same distributions (around 0.0016s) because the random pivot bypassed the worst-case splits.

During the testing of the "Repeated Array," both algorithms achieved their fastest times (fractions of a millisecond). Because the array consisted of only 5 unique repeated numbers, the middle partition of the algorithm absorbed roughly 200 duplicate elements in a single pass. Since all items in the middle list are identical and require no further sorting, the algorithm effectively skipped hundreds of recursive steps. This demonstrates how isolating duplicate elements into a dedicated partition dynamically optimizes the algorithm's performance on arrays with repeated elements.